

0.70.3. Plus fluide : la lettre au pixel près — en simple

Le 0.70 en une ligne : `glisse_croix(0, px, py, ALPHABET[0], 1)`.

1. Ce qu'il faut comprendre

C'est le 0.70, en plus simple : les quatre `if` des flèches et le `sprite` sont remplacés par une ligne, `glisse_croix` (le 0.73).

La vitesse, 1 : un pixel par image, comme au 0.70.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1592 octets** de cartouche.

1. UNE VARIABLE — PX

```
uint8_t px = 76;    // en pixels : le milieu
```

2. UNE VARIABLE — PY

```
uint8_t py = 68;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

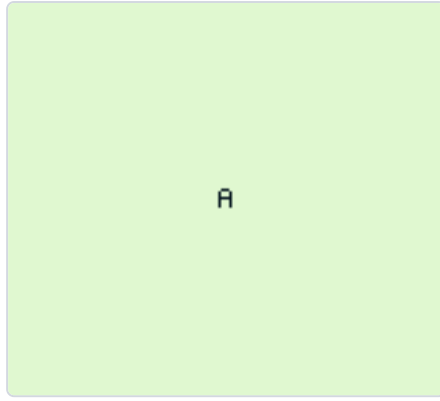
```
int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[0], 1);    // tout le 0.70
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.70 : un A qui glisse pixel par pixel avec les flèches.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 30 pixels en 30 images

Ensuite : **0.71. La croix en une ligne : deplace_croix** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++